

ViolationIQ: Adaptive Multi-Expert AI Evidence Copilot for Automated Traffic Violation Identification Using Computer Vision

Prototype Proposal and System Framework

Computer Vision Based Traffic Enforcement Evidence Generation System

Problem Statement: Automated Photo Identification and Classification for Traffic Violations Using Computer Vision

Abstract—The rapid deployment of traffic surveillance cameras has increased the volume of road images and videos generated every day. Manual inspection of this data for traffic violations is labor-intensive, time-consuming, and prone to inconsistent decisions. This paper presents ViolationIQ, an adaptive multi-expert computer vision framework for automated traffic violation evidence generation. The system detects road users, identifies violation-related visual evidence, classifies the violation context, and produces clean annotated evidence for review. Instead of acting as a blind challan generator, ViolationIQ is designed as an AI evidence copilot. It uses separate expert modules for helmet and rider detection, number plate localization, safe OCR, red-light video evidence, and traffic signboard context. The system combines YOLO-based object detection, OpenCV-based image/video processing, OCR validation, rule-based reasoning, evidence reliability scoring, and a safety layer that sends uncertain cases to manual review. Experimental development results include a traffic YOLO11s model with mAP50 around 0.928, a helmet/rider YOLO11s model with mAP50 around 0.701, and a dedicated plate YOLO11s model with validation mAP50 around 0.924. The final prototype generates processed MP4 demo videos, annotated evidence panels, JSON/CSV reports, and GitHub-ready documentation. The proposed solution is practical for real enforcement scenarios because it focuses not only on detection, but also on evidence reliability, reviewability, and safety-aware decision support.

Index Terms—Computer Vision, Traffic Violation Detection, YOLO, Helmet Detection, Number Plate Detection, OCR, Red-Light Violation, Evidence Generation, Intelligent Transportation Systems.

I. INTRODUCTION

Traffic enforcement agencies increasingly rely on surveillance cameras, CCTV systems, red-light cameras, and automated road monitoring infrastructure. These systems continuously generate large volumes of images and videos. However, identifying traffic violations from this evidence manually is slow and difficult to scale. Human review may also become inconsistent when there are many cameras, varying image qualities, poor lighting conditions, occlusion, and crowded traffic scenes.

The problem statement requires an intelligent system capable of automatically processing traffic images, detecting vehicles and road users, identifying traffic violations, classifying the type of violation, and generating annotated evidence for further review. A practical solution must work under real-world conditions such as low light, motion blur, small number

plates, different camera angles, occlusion, and traffic density variations.

This paper proposes **ViolationIQ**, an adaptive multi-expert AI evidence copilot for traffic enforcement. The system is designed to support human officers by generating clean evidence, not by issuing unsafe automatic challans. The core idea is to use multiple specialized computer vision modules rather than relying on one model for every violation type.

A. Motivation

Traffic violations such as riding without a helmet, crossing a red signal, entering restricted zones, or ignoring traffic signs require different types of visual evidence. A helmet violation may be detected from a single image if the rider and helmet status are clear. A red-light violation requires signal color, vehicle detection, and stop-line crossing evidence. Number plate recognition requires reliable OCR and should not be forced when the plate is blurred. Therefore, the system must reason about the quality and meaning of evidence.

B. Contribution

The main contributions of this project are:

- An adaptive multi-expert architecture for traffic violation evidence generation.
- Rider-wise helmet compliance detection and evidence panel generation.
- Dedicated number plate localization with safe OCR fallback.
- Red-light video evidence using signal color detection, vehicle detection, and virtual stop-line crossing.
- Signboard context evidence for traffic signs such as no-entry, no-stopping, stop, and turn restrictions.
- A safety layer that prevents forced OCR and routes uncertain cases to manual review.
- GitHub-ready prototype with source code, architecture diagrams, reports, outputs, and processed MP4 demo video.

II. PROBLEM UNDERSTANDING

The given problem is not merely an object detection task. A practical traffic enforcement system must answer three important questions:

- 1) **What is visible?** The system must detect vehicles, riders, helmets, number plates, traffic lights, and traffic signs.

- 2) **What does it mean?** The system must classify the violation context, such as no helmet, red-light crossing, or signboard restriction.
- 3) **Is the evidence reliable?** The system must decide whether the evidence is strong enough for review or whether it should be marked uncertain.

A simple detector may identify a plate or rider, but a real-world system must also handle false positives, weak OCR, unclear lighting, occlusion, and camera calibration limitations. Therefore, ViolationIQ is designed around evidence generation and safety-aware review.

A. Challenges in Automated Traffic Evidence

The practical challenges include:

- small and blurred number plates,
- partial occlusion of riders and vehicles,
- low-light scenes and glare,
- camera angle variation,
- rain, shadows, and motion blur,
- visually similar helmet and non-helmet cases,
- single-frame ambiguity in red-light evidence,
- need for manual review before enforcement.

These issues make it necessary to build an evidence-aware framework instead of a pure detector.

III. PROPOSED SYSTEM OVERVIEW

ViolationIQ is a modular computer vision framework that takes traffic images or videos as input and produces annotated evidence outputs. The system uses an adaptive router to select the appropriate expert module based on input type and scene context.

The implemented system covers:

- helmet and rider detection,
- rider-wise evidence generation,
- dedicated number plate detection,
- safe OCR/ANPR fallback,
- red-light video evidence,
- signboard context evidence,
- structured JSON/CSV reporting,
- processed MP4 demo outputs.

The system intentionally avoids overclaiming. It does not issue final challans automatically. Instead, it creates review-ready evidence and marks uncertain cases for human review.

A. Design Philosophy

ViolationIQ follows four design principles:

- 1) **Modularity:** each violation type is handled by a specialized expert.
- 2) **Explainability:** every output contains evidence, significance, reliability, and priority.
- 3) **Safety:** uncertain cases are not forced into final decisions.
- 4) **Scalability:** evidence and reports are generated automatically in structured form.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the overall architecture of ViolationIQ.

A. Why Adaptive Architecture Was Used

Traffic violations are visually different. A helmet violation depends on rider-face-helmet association. A red-light violation depends on signal state and vehicle crossing. A number plate module needs plate localization and OCR validation. A signboard module provides road rule context. Hence, a single monolithic model would be less transparent and harder to debug. The adaptive architecture improves clarity, modularity, and reliability.

V. DATASETS USED

The project uses task-specific datasets. Table I lists the datasets used in training and demo generation.

A. Need for Multiple Datasets

Different modules require different supervision. Helmet detection requires rider-level labeling. Number plate detection requires accurate small-object localization. Red-light evidence requires video-based reasoning. Signboard context requires traffic sign classes. Therefore, multiple datasets were required for a complete solution framework.

VI. TECHNOLOGY STACK

Table II summarizes the major tools and their role.

VII. MODELS AND EXPERIMENTAL RESULTS

Table III presents the recorded experimental results.

The helmet/rider model performance is moderate compared with the traffic and plate models because rider scenes contain occlusion, motion blur, partial faces, small helmets, and camera-angle variation. Therefore, the safety layer is important for practical deployment.

A. Evaluation Metrics

The following evaluation indicators were used during development:

- **mAP50:** mean average precision at IoU threshold 0.50. It indicates object detection performance when a predicted box sufficiently overlaps the ground truth.
- **mAP50-95:** mean average precision averaged over IoU thresholds from 0.50 to 0.95. This is stricter than mAP50.
- **Confidence Score:** model confidence for each detected object.
- **OCR Confidence:** reliability score of the OCR output.
- **Evidence Priority:** final review priority generated by the reasoning layer.

VIII. METHODOLOGY

The complete methodology is divided into five stages: data preparation, model training, detection inference, evidence reasoning, and output generation.

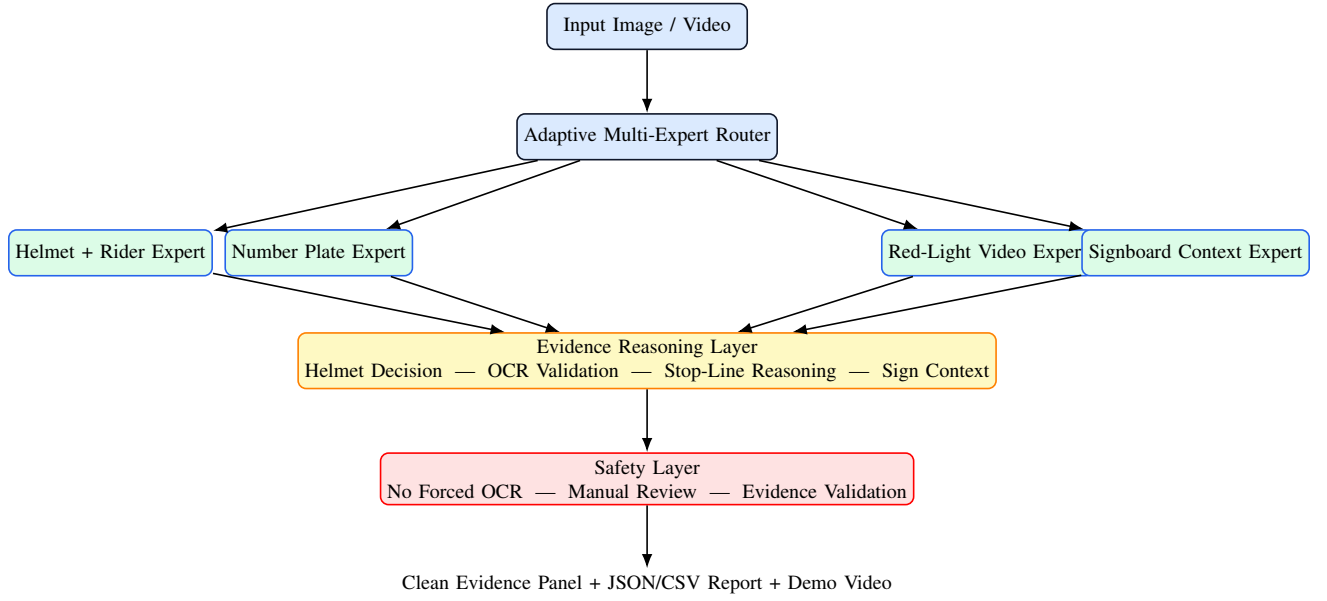


Fig. 1: ViolationIQ adaptive multi-expert architecture. The router sends each input to specialized expert modules, while the reasoning and safety layers convert detections into review-ready evidence.

TABLE I: Datasets Used in ViolationIQ

Module	Dataset Used	Purpose
Traffic / Signboard Expert	guisahanes/traffic-violation-detection	Used for detecting vehicles, traffic lights, traffic signs, no-entry, no-stopping, stop signs, and turn-context evidence.
Helmet + Rider Expert	aryanvaid13/indian-helmet-detection	Used for detecting riders and classifying helmet, no-helmet, bad helmet, and number plate class.
Dedicated Plate Expert	Large YOLO plate dataset with 10,125 images	Used for dedicated number plate localization and plate crop extraction.
Red-Light Evidence Module	farzadnekouei/license-plate-recognition	Used for red-light video evidence, signal color detection, vehicle crossing, and virtual stop-line demonstration.
Final Demo Video	Private uploaded dataset Videofbike	Used only for the final processed helmet violation MP4 demo. It was not used for training.

TABLE II: Technology Stack

Technology	Role
Python	Main implementation language.
YOLO11s	Object detection for traffic, helmet, rider, and plate modules.
OpenCV	Video processing, drawing evidence panels, color thresholding, and frame extraction.
EasyOCR	OCR layer for reading plate crops.
Pandas	CSV report generation.
JSON	Structured evidence reports.
Kaggle	Training and experimental environment.
GitHub	Final repository and demo presentation.

A. Stage 1: Data Preparation

Each dataset was inspected and converted into the required YOLO format where necessary. The dataset YAML files were corrected to ensure that training and validation paths were properly linked. For the helmet module, the class map was manually verified because the project required rider-wise association.

B. Stage 2: Model Training

YOLO11s models were trained for traffic/sign context, helmet/rider detection, and number plate localization. The training was performed in Kaggle using GPU acceleration. After training, the best weights were saved and later used as checkpoint models.

C. Stage 3: Inference

During inference, each image or frame is passed through the selected expert model. The system records bounding box coordinates, class labels, confidence values, and relevant metadata.

D. Stage 4: Evidence Reasoning

Raw detections are converted into evidence decisions. For helmet evidence, the system associates face/helmet detections with rider boxes. For red-light evidence, signal color and vehicle crossing are combined. For plate evidence, OCR is validated before displaying the plate number.

TABLE III: Model Results Recorded During Development

Model / Module	Recorded Result
Traffic YOLO11s model	mAP50 around 0.928 and mAP50-95 around 0.808.
Helmet/Rider YOLO11s model	mAP50 around 0.701 and mAP50-95 around 0.32.
Dedicated Plate YOLO11s model	Validation mAP50 around 0.924 and mAP50-95 around 0.548.
Plate dataset split	7057 train images, 2048 validation images, and 1020 test images.
Red-Light Video Evidence	Signal color detection, vehicle detection, virtual stop-line crossing, and manual-review safety.
Uploaded Bike Helmet Demo	Processed MP4 demo with rider-wise helmet violation evidence.

E. Stage 5: Output Generation

The final stage generates annotated evidence images, processed MP4 videos, JSON reports, CSV summaries, and final showcase outputs for GitHub and demonstration.

IX. MODULE 1: ADAPTIVE MULTI-EXPERT ROUTER

The router decides which module should process an input. For example, a video input may go to the red-light or helmet video module, while a still image containing traffic signs may go to the signboard module.

Algorithm 1 Adaptive Multi-Expert Routing

Require: Input image or video I
Ensure: Selected expert module and evidence output

- 1: **if** I is a video **then**
- 2: Extract frames from I
- 3: Check signal, vehicle, rider, and scene context
- 4: **if** red-light scene is detected **then**
- 5: Route to Red-Light Video Expert
- 6: **else**
- 7: Route to Helmet/Rider Video Expert
- 8: **end if**
- 9: **else**
- 10: Run lightweight context detection
- 11: **if** rider or helmet objects are detected **then**
- 12: Route to Helmet + Rider Expert
- 13: **end if**
- 14: **if** plate region is detected **then**
- 15: Route to Number Plate Expert
- 16: **end if**
- 17: **if** traffic sign is detected **then**
- 18: Route to Signboard Context Expert
- 19: **end if**
- 20: **end if**
- 21: Send detections to Evidence Reasoning Layer
- 22: Apply Safety Layer
- 23: **return** Clean evidence output

A. Benefits of Routing

The router reduces unnecessary computation and improves interpretability. Each expert module produces outputs that are meaningful for its task. This makes debugging easier and makes the final evidence more transparent.

X. MODULE 2: HELMET AND RIDER EVIDENCE

The helmet module detects riders and classifies helmet compliance in a rider-wise format. The key problem is not just detecting a helmet, but associating helmet or no-helmet evidence with the correct rider.

A. Helmet Classes

TABLE IV: Helmet Dataset Classes

Class ID	Class Name
0	numberPlate
1	faceWithNoHelmet
2	faceWithGoodHelmet
3	faceWithBadHelmet
4	rider

B. Rider-wise Association Algorithm

For each detected rider bounding box, the system checks whether the center of a face-with-no-helmet, face-with-good-helmet, or face-with-bad-helmet detection lies inside the rider box. Based on this association, the rider is classified as *No Helmet*, *Helmet OK*, *Improper Helmet*, or *Manual Review*.

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2} \quad (1)$$

A face detection is associated with a rider if:

$$x_r^1 \leq c_x \leq x_r^2 \quad \text{and} \quad y_r^1 \leq c_y \leq y_r^2 \quad (2)$$

Algorithm 2 Rider-wise Helmet Compliance Decision

Require: Rider boxes R , helmet face boxes H , no-helmet boxes N , bad-helmet boxes B
Ensure: Rider-wise helmet status

- 1: **for** each rider $r \in R$ **do**
- 2: status $\leftarrow$ Manual Review
- 3: **if** center of any $n \in N$ lies inside r **then**
- 4: status $\leftarrow$ No Helmet
- 5: **else if** center of any $b \in B$ lies inside r **then**
- 6: status $\leftarrow$ Improper Helmet
- 7: **else if** center of any $h \in H$ lies inside r **then**
- 8: status $\leftarrow$ Helmet OK
- 9: **end if**
- 10: Assign rider ID such as R1, R2, R3
- 11: **end for**
- 12: Generate annotated rider-wise evidence panel

C. Helmet Output

The final helmet output includes:

- rider-wise bounding boxes,
- helmet/no-helmet status,
- evidence summary panel,
- reliability statement,
- priority label,
- processed MP4 demo video.

XI. MODULE 3: NUMBER PLATE DETECTION AND SAFE OCR

The plate module uses a dedicated detector for number plate localization. OCR is then applied only when the crop is readable enough.

A. Plate Detection

The dedicated plate detector was trained on a larger plate dataset containing 10,125 images. It achieved validation mAP50 around 0.924 and mAP50-95 around 0.548. This model is used for plate localization, not for final legal verification.

B. Safe OCR

OCR systems can generate incorrect plate numbers when plates are blurred, tilted, small, or overexposed. Therefore, ViolationIQ follows a safe OCR policy.

Algorithm 3 Safe OCR / ANPR Validation

Require: Plate crop P

Ensure: Plate text or manual review label

- 1: Enhance crop using resizing and contrast improvement
 - 2: Compute readability score
 - 3: Run OCR on enhanced crop
 - 4: Clean OCR text by removing invalid characters
 - 5: **if** OCR confidence is high and text format is valid **then**
 - 6: Display plate text
 - 7: **else**
 - 8: Display UNREADABLE
 - 9: Mark for manual review
 - 10: **end if**
-

This prevents forced OCR and reduces the risk of wrong challans.

XII. MODULE 4: RED-LIGHT VIDEO EVIDENCE

The red-light module processes traffic video frames and produces evidence based on signal state and vehicle crossing.

A. Signal Color Detection

OpenCV HSV thresholding is used to detect signal color from a defined signal region. The system classifies signal state as red, green, yellow, or unknown.

Algorithm 4 Red-Light Evidence Decision

Require: Video frame F , signal ROI, virtual stop-line L

Ensure: Red-light evidence decision

- 1: Detect signal color using HSV thresholding
 - 2: Detect vehicles using YOLO
 - 3: **for** each vehicle box v **do**
 - 4: **if** bottom of v crosses stop-line L **then**
 - 5: Mark v as crossing candidate
 - 6: **end if**
 - 7: **end for**
 - 8: **if** signal is red and crossing candidate exists **then**
 - 9: decision $\leftarrow$ Possible Red-Light Violation
 - 10: priority $\leftarrow$ High
 - 11: **else**
 - 12: decision $\leftarrow$ No confirmed evidence or manual review
 - 13: **end if**
 - 14: Generate evidence frame and report
-

B. Vehicle Crossing Logic

A virtual stop-line is drawn in the frame. If the signal is red and a detected vehicle crosses the virtual stop-line, the system marks the frame as a red-light violation evidence candidate.

C. Output

The red-light output contains:

- red signal badge,
- virtual stop-line,
- evidence vehicle bounding box,
- significance statement,
- reliability statement,
- manual-review warning.

XIII. MODULE 5: SIGNBOARD CONTEXT EVIDENCE

The signboard context module detects traffic signs and generates context evidence. It does not directly issue enforcement decisions from a sign alone. Instead, it provides evidence that a traffic rule exists in that scene.

Supported contexts include:

- no-entry sign,
- no-stopping sign,
- stop sign,
- turn restriction signs,
- speed-limit sign context,
- traffic-light context.

For example, if a no-entry sign is detected, the system outputs a no-entry context panel. Final enforcement may still need vehicle direction or route evidence depending on the rule.

XIV. EVIDENCE REASONING AND SAFETY LAYER

The evidence reasoning layer converts detections into meaningful outputs. The safety layer prevents unsafe claims.

TABLE V: Evidence Priority Meaning

Priority	Meaning
High	Clear evidence candidate requiring human review before challan.
Medium	Scene partially clear, manual review recommended.
Low	No strong violation evidence visible.

A. Evidence Priority

B. Manual Review Conditions

ViolationIQ routes cases to manual review when:

- OCR confidence is weak,
- plate is unreadable,
- signal is unclear,
- evidence is visually ambiguous,
- detection confidence is low,
- camera calibration is required.

This safety-first approach makes the system more practical and responsible.

XV. EVIDENCE OUTPUT DESIGN

A major focus of ViolationIQ is clean evidence presentation. The evidence output is designed for review by enforcement officials, not only for model debugging.

A. Helmet Evidence Panel

The helmet evidence panel contains:

- original traffic frame,
- rider-wise bounding boxes,
- short rider IDs such as R1, R2, R3,
- side panel with rider status,
- no-helmet violation count,
- reliability and significance statement,
- manual-review reminder.

B. Red-Light Evidence Panel

The red-light evidence panel contains:

- red signal indicator,
- virtual stop-line,
- evidence vehicle box,
- frame number,
- vehicle crossing count,
- priority statement,
- temporal/manual-review safety note.

C. Signboard Context Panel

The signboard context panel contains:

- detected signboard class,
- bounding box around the sign,
- detected traffic context,
- enforcement rule explanation,
- manual-review note if additional tracking is required.

XVI. IMPLEMENTED DELIVERABLES

Table VI lists only the deliverables implemented in the prototype.

XVII. REPOSITORY AND OUTPUT STRUCTURE

The final repository contains source code, architecture diagrams, reports, demo videos, and output evidence.

```
ViolationIQ/
|-- README.md
|-- requirements.txt
|-- src/
|   |-- adaptive_router.py
|   |-- helmet_plate_module.py
|   |-- redlight_module.py
|   |-- signboard_context_module.py
|   |-- safety_utils.py
|-- config/
|-- architecture/
|-- reports/
|-- outputs/
|   |-- helmet_plate/
|   |-- redlight/
|   |-- signboard_context/
|   |-- video/
|   |-- FINAL_SHOWCASE/
|-- demo/
|   |-- violationiq_bike_helmet_demo.mp4
```

XVIII. HOW THE SYSTEM ADDRESSES THE PROBLEM STATEMENT

Table VII maps the problem requirements to the implemented solution.

XIX. DEPLOYMENT WORKFLOW

In a practical deployment scenario, the traffic department can use ViolationIQ as follows:

- 1) A traffic image or video is uploaded from a surveillance camera.
- 2) The adaptive router identifies the type of input and scene.
- 3) The suitable expert module processes the evidence.
- 4) The reasoning layer generates the violation context.
- 5) The safety layer checks reliability.
- 6) A clean annotated evidence panel is created.
- 7) JSON/CSV reports are stored for audit.
- 8) A human reviewer verifies the evidence before enforcement action.

This workflow reduces manual workload while keeping the final decision under human supervision.

A. Expected Real-World Impact

The expected impact includes:

- faster evidence preparation,
- reduced manual review burden,
- better consistency in evidence formatting,
- improved auditability through JSON/CSV reports,
- safer challan workflow through manual-review fallback,
- scalable support for multiple camera feeds.

TABLE VI: Implemented Deliverables

No.	Implemented Deliverable	Output Produced
1	Adaptive Multi-Expert Router	Routes image/video input toward the suitable evidence module.
2	Helmet + Rider Detection	Detects riders and classifies helmet/no-helmet status.
3	Rider-wise Evidence Panel	Generates clean R1, R2, R3 style rider evidence.
4	Dedicated Number Plate Detection	Detects number plate regions using a separate trained detector.
5	Safe OCR / ANPR Fallback	Shows plate number only when reliable and routes weak OCR to review.
6	Red-Light Video Evidence	Detects signal color, vehicle presence, and virtual stop-line crossing evidence.
7	Signboard Context Evidence	Detects traffic sign context such as no-entry, no-stopping, stop, and turn signs.
8	Final Demo Videos	Stores processed MP4 demo videos inside <code>demo/</code> and <code>outputs/video/</code> .
9	JSON / CSV Reports	Saves structured evidence reports for review.
10	Architecture and Documentation	Includes architecture diagram, modular source code, configuration files, reports, and README.

TABLE VII: Problem Requirement Mapping

Problem Requirement	ViolationIQ Solution
Automatically process traffic images/videos	Uses OpenCV and Python pipeline for images and frame-wise video processing.
Detect vehicles and road users	Uses YOLO-based expert models for vehicles, riders, helmets, plates, and signs.
Identify traffic violations	Detects helmet violation evidence, red-light crossing evidence, and signboard context.
Classify type of violation	Produces module-specific classification such as helmet violation, red-light evidence, plate evidence, or sign context.
Generate annotated evidence	Creates clean evidence panels with boxes, labels, summaries, reliability, and priority.
Handle unreliable evidence	Uses safe OCR, reliability checks, and manual-review fallback.
Support scalability	Uses modular expert design and structured JSON/CSV reports.

XX. LIMITATIONS AND DEPLOYMENT CONSIDERATIONS

The current prototype demonstrates practical modules, but real-world deployment requires careful integration. Number plate OCR can fail in very blurred or low-resolution frames. Helmet detection may be affected by occlusion or crowded traffic. Red-light evidence depends on camera viewpoint and stop-line placement. Signboard context indicates road rule presence, but final enforcement may require additional evidence depending on the rule.

Before deployment, the system should be integrated with camera-specific calibration, audit logs, officer review workflow, privacy safeguards, and periodic model revalidation.

A. Responsible AI Considerations

Traffic enforcement is a sensitive domain because incorrect decisions can directly affect citizens. Therefore, ViolationIQ avoids unsafe automatic challans. The system is designed to produce evidence candidates. Human officers remain responsible for final enforcement decisions.

XXI. FUTURE SCOPE

Future improvements can include:

- stronger low-light enhancement,
- better Indian plate OCR model,
- camera calibration dashboard,
- centralized officer review dashboard,
- temporal tracking for more complex violations,
- city-scale camera feed integration,
- active learning from reviewed evidence.

XXII. CONCLUSION

This paper presented ViolationIQ, an adaptive multi-expert AI evidence copilot for automated traffic violation identification using computer vision. The system addresses the challenge by detecting road users and traffic context, identifying violation evidence, classifying the evidence type, and generating annotated outputs for review.

The prototype implements helmet and rider detection, rider-wise evidence panels, dedicated number plate detection, safe OCR fallback, red-light video evidence, signboard context evidence, JSON/CSV reporting, and processed MP4 demo outputs. The system uses YOLO-based detection, OpenCV-based reasoning, OCR validation, and a safety-first manual review layer.

The main strength of ViolationIQ is its responsible design. It does not act as an unsafe automatic challan generator. Instead, it supports traffic enforcement teams with clean, structured, and review-ready evidence while keeping uncertain cases under human review.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [3] J. Memon, M. Sami, R. A. Khan, and M. Uddin, "Handwritten Optical Character Recognition: A Comprehensive Systematic Literature Review," *IEEE Access*, vol. 8, pp. 142642–142668, 2020.
- [4] M. A. Boukerche and J. Wang, "Machine Learning-Based Traffic Prediction Models for Intelligent Transportation Systems," *Computer Networks*, vol. 181, 2020.
- [5] Ultralytics, "YOLO Documentation," 2024. [Online]. Available: <https://docs.ultralytics.com/>
- [6] JaidedAI, "EasyOCR," 2024. [Online]. Available: <https://github.com/JaidedAI/EasyOCR>